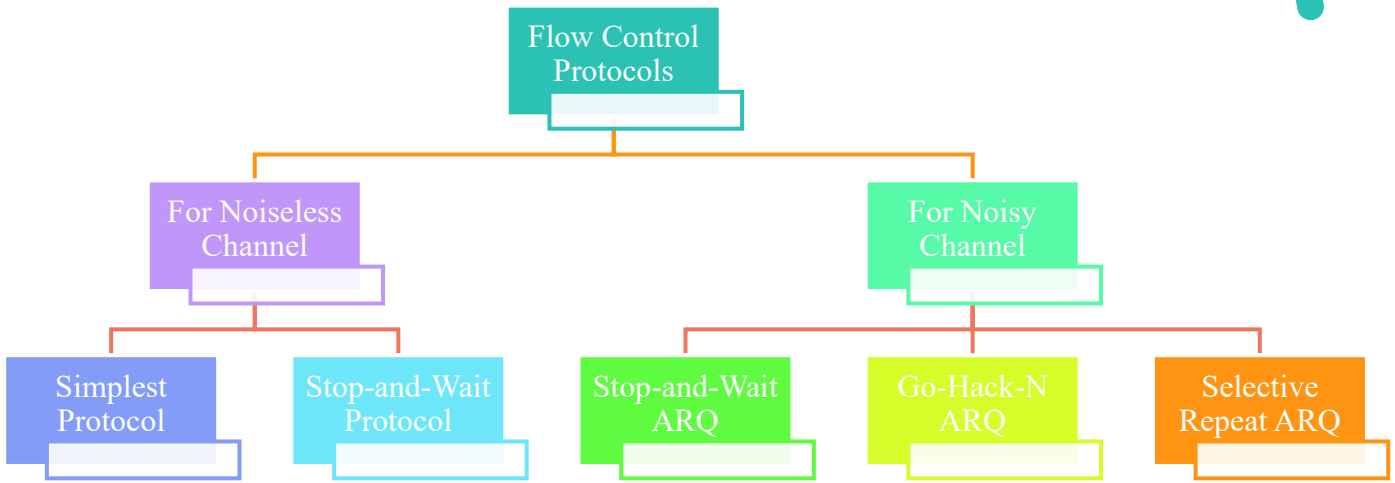
Flow Control

- Flow control refers to a set of procedures used to restrict the amount of data that the sender can send before waiting for acknowledgment.
- In most protocols, flow control is a set of procedures that tells the sender how much data it can transmit before it must wait for an acknowledgment from the receiver.
- The flow of data must not be allowed to overwhelm the receiver.
- Any receiving device has a limited speed at which it can process incoming data and a limited amount of memory in which to store incoming data.
- The receiving device must be able to inform the sending device before those limits are reached and to request that the transmitting device send fewer frames or stop temporarily.

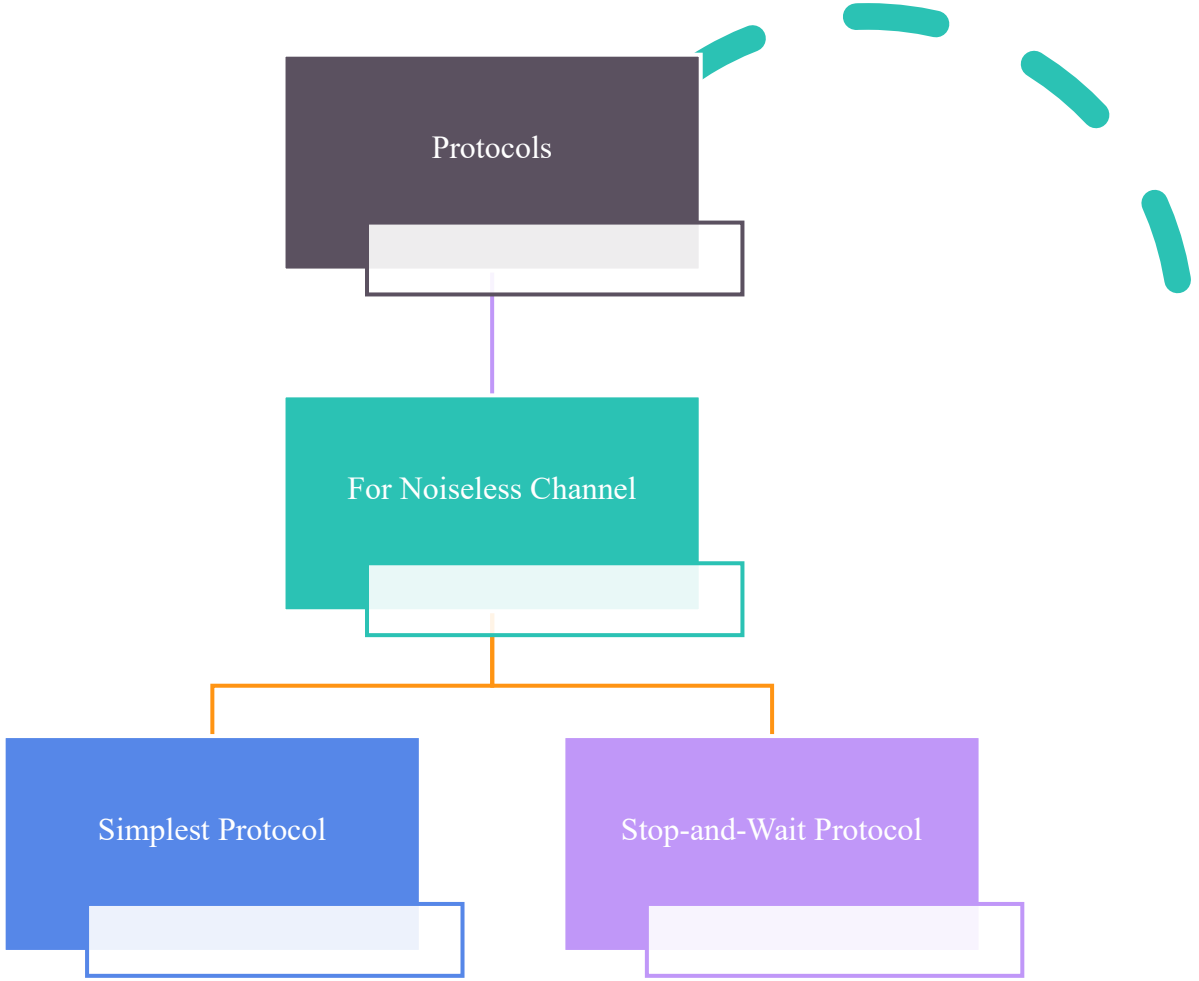
Flow Control

- Incoming data must be checked and processed before they can be used.
- The rate of such processing is often slower than the rate of transmission.
- For this reason, each receiving device has a block of memory, called a buffer, reserved for storing incoming data until they are processed.
- If the buffer begins to fill up, the receiver must be able to tell the sender to halt transmission until it is once again able to receive.

Types Of Flow Control



Types Of Flow Control



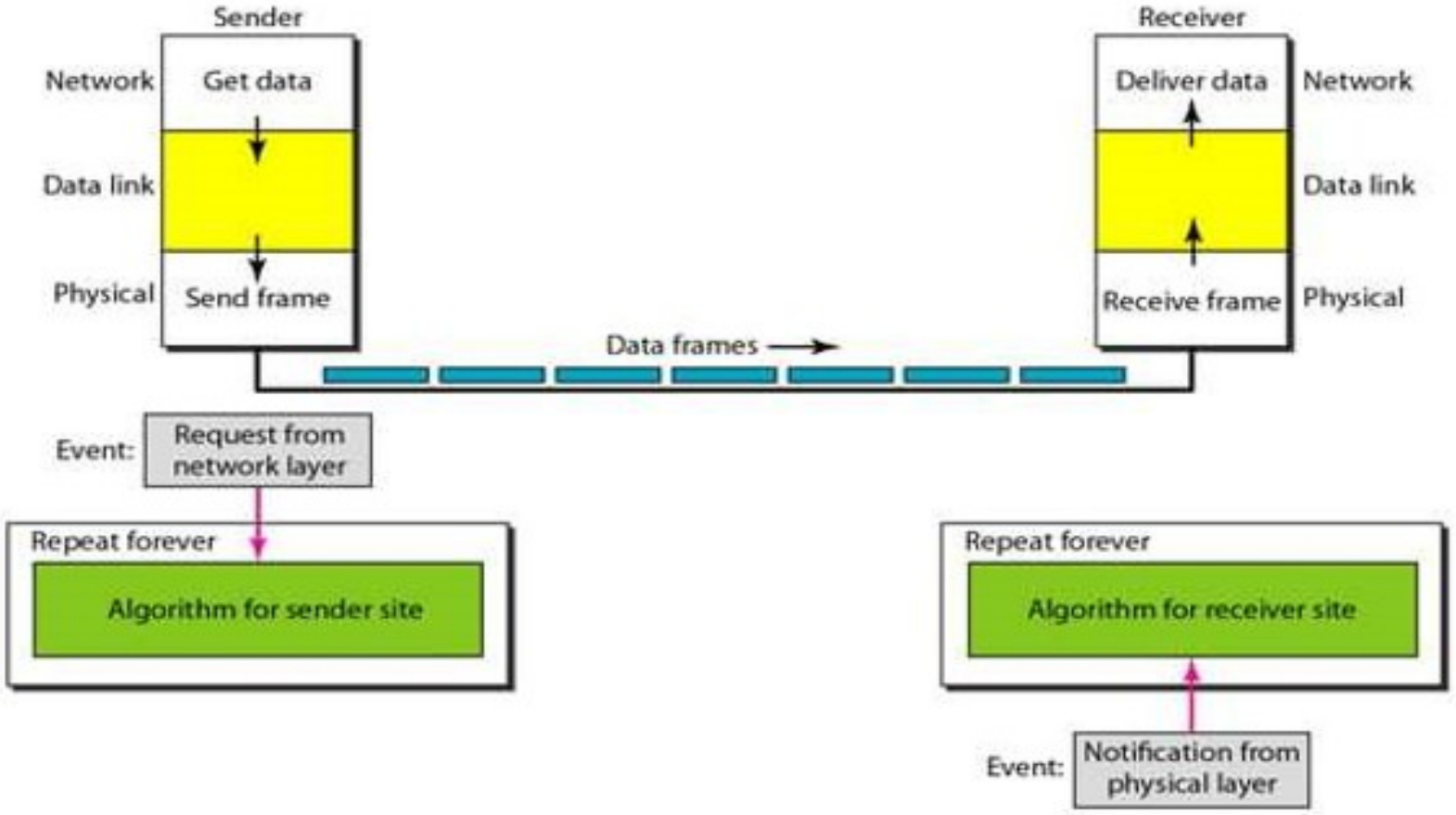
Simplest Protocol

- Our first protocol, which we call the Simplest Protocol for lack of any other name, is one that has no flow control.
- Like other protocols we will discuss in this chapter, it is a unidirectional protocol in which data frames are traveling in only one direction from sender to receiver.
- We assume that the receiver can immediately handle any frame it receives with a processing time that is small enough to be negligible.
- The data link layer of the receiver immediately removes the header from the frame and hands the data packet to its network layer, which can also accept the packet immediately.
- In other words, the receiver can never be overwhelmed with incoming frames.

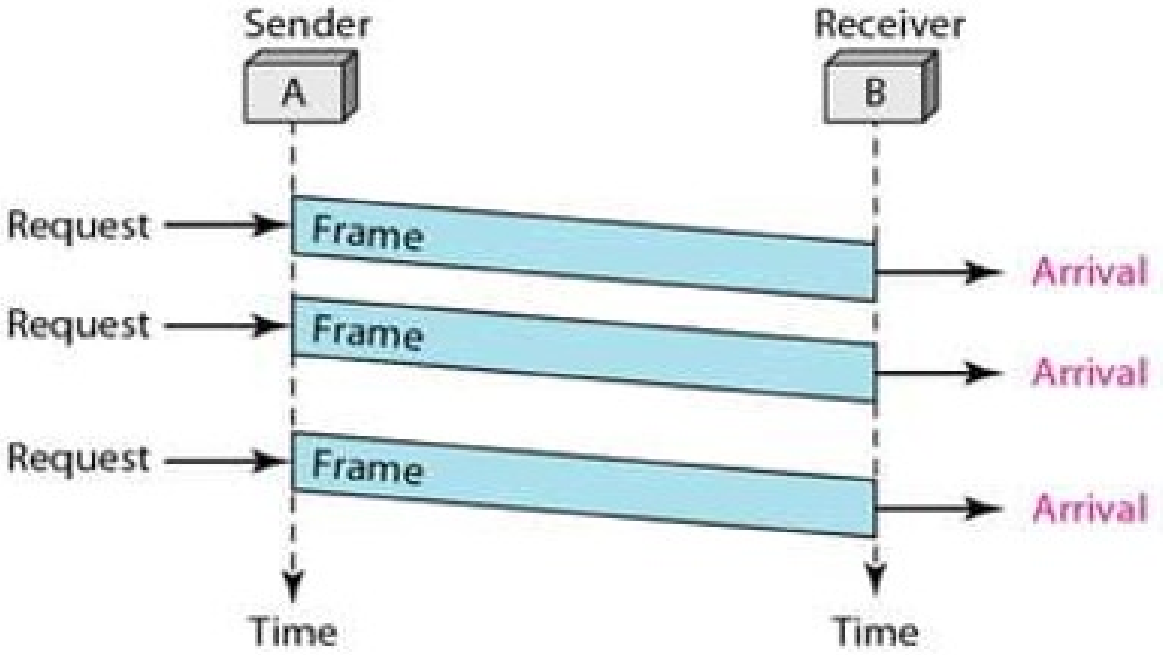
Simplest Protocol- Design

- There is no need for flow control in this scheme.
- The data link layer at the sender site gets data from its network layer, makes a frame out of the data, and sends it.
- The data link layer at the receiver site receives a frame from its physical layer, extracts data from the frame, and delivers the data to its network layer.
- The data link layers of the sender and receiver provide transmission services for their network layers.
- The data link layers use the services provided by their physical layers (such as signaling, multiplexing, and so on) for the physical transmission of bits.

Simplest Protocol- Design



Flow Design


Sender
A
Receiver
B
Request
Frame
Arrival
Request
Frame
Arrival
Request
Frame
Arrival
Time
Time

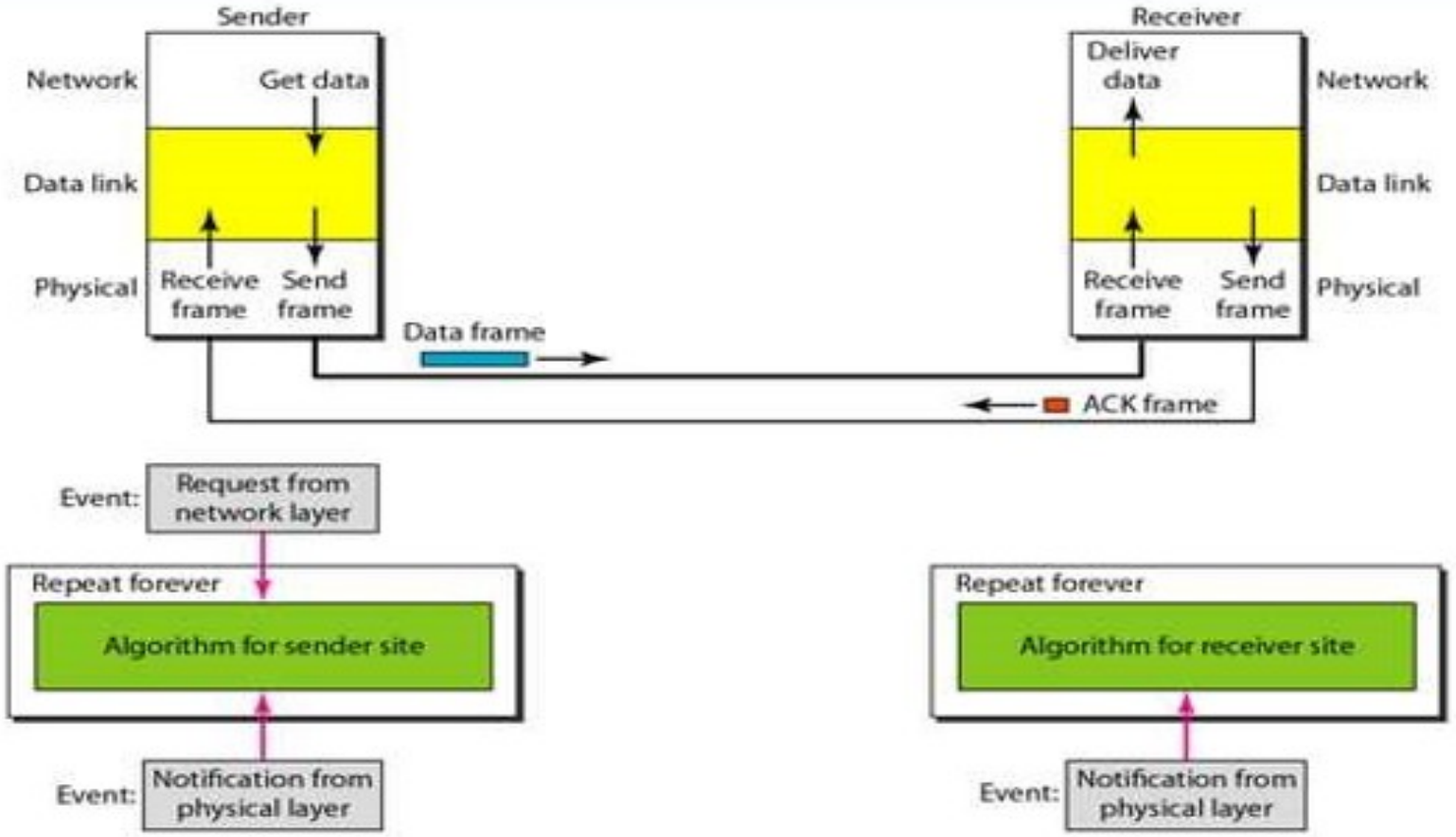
Stop and Wait Protocol

- If data frames arrive at the receiver site faster than they can be processed, the frames must be stored until their use.
- Normally, the receiver does not have enough storage space, especially if it is receiving data from many sources.
- This may result in either the discarding of frames or denial of service.
- To prevent the receiver from becoming overwhelmed with frames, we somehow need to tell the sender to slow down.
- There must be feedback from the receiver to the sender.

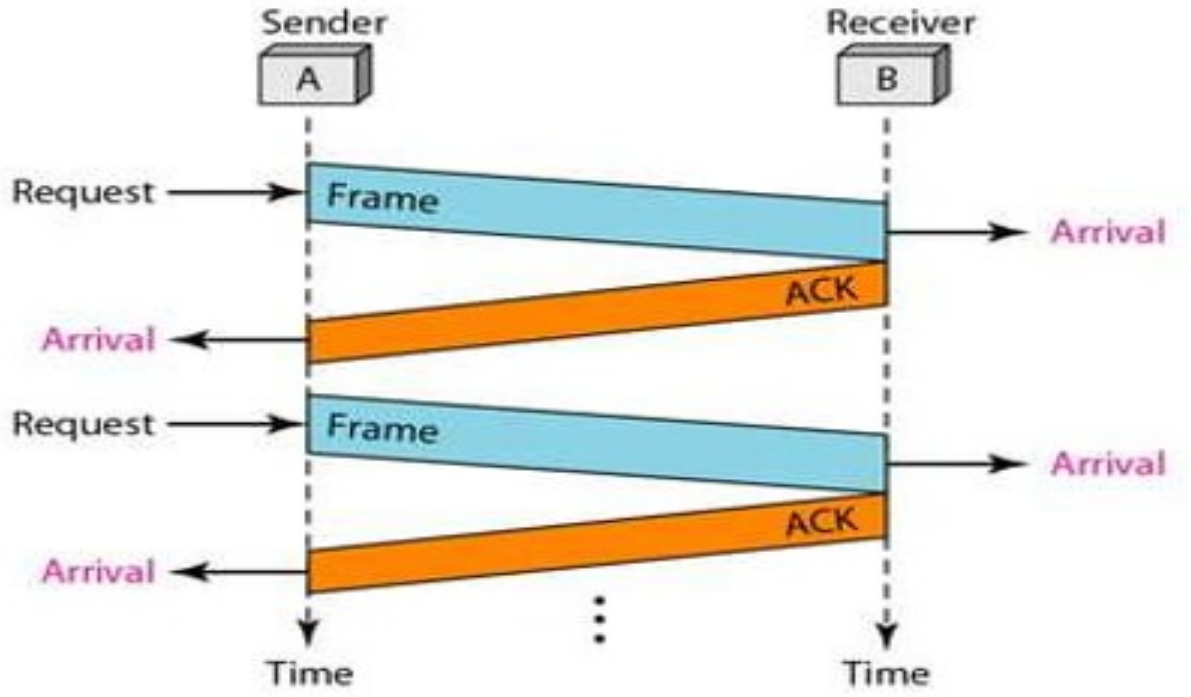
Stop and Wait Protocol

- The protocol we discuss now is called the Stop-and-Wait Protocol because the sender sends one frame, stops until it receives confirmation from the receiver (okay to go ahead), and then sends the next frame.
- We still have unidirectional communication for data frames, but auxiliary ACK frames (simple tokens of acknowledgment) travel from the other direction.
- We add flow control to our previous protocol.

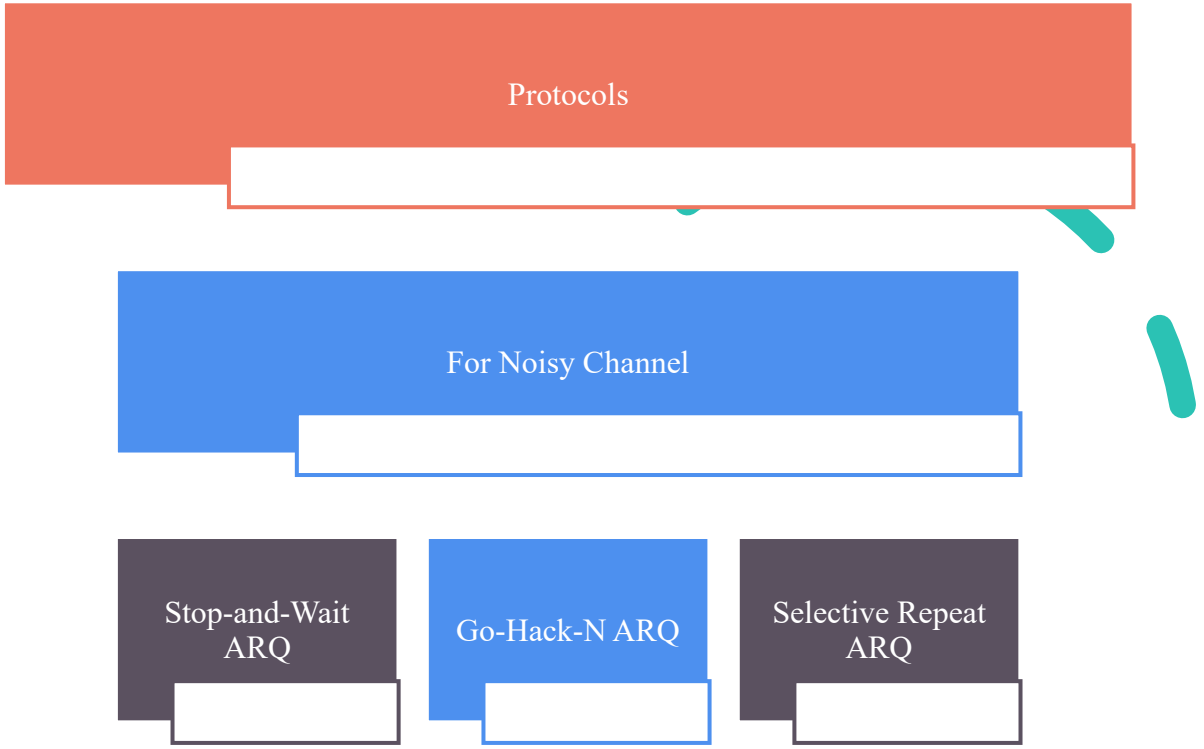
Stop and Wait Protocol- Design



Stop and Wait Protocol- Flow Control



Types Of Flow Control



Stop and Wait with ARQ Protocol

- Our first protocol, called the Stop-and-Wait Automatic Repeat Request (Stop-and Wait ARQ), adds a simple error control mechanism to the Stop-and-Wait Protocol.
- Let us see how this protocol detects and corrects errors.
- Error correction in Stop-and-Wait ARQ is done by keeping a copy of the sent frame and retransmitting of the frame when the timer expires.
- To detect and correct corrupted frames, we need to add redundancy bits to our data frame When the frame arrives at the receiver site, it is checked and if it is corrupted, it is silently discarded.

Stop and Wait with ARQ Protocol

- The detection of errors in this protocol is manifested by the silence of the receiver.
- Lost frames are more difficult to handle than corrupted ones. In our previous protocols, there was no way to identify a frame.
- The received frame could be the correct one, or a duplicate, or a frame out of order. The solution is to number the frames.
- When the receiver receives a data frame that is out of order, this means that frames were either lost or duplicated.

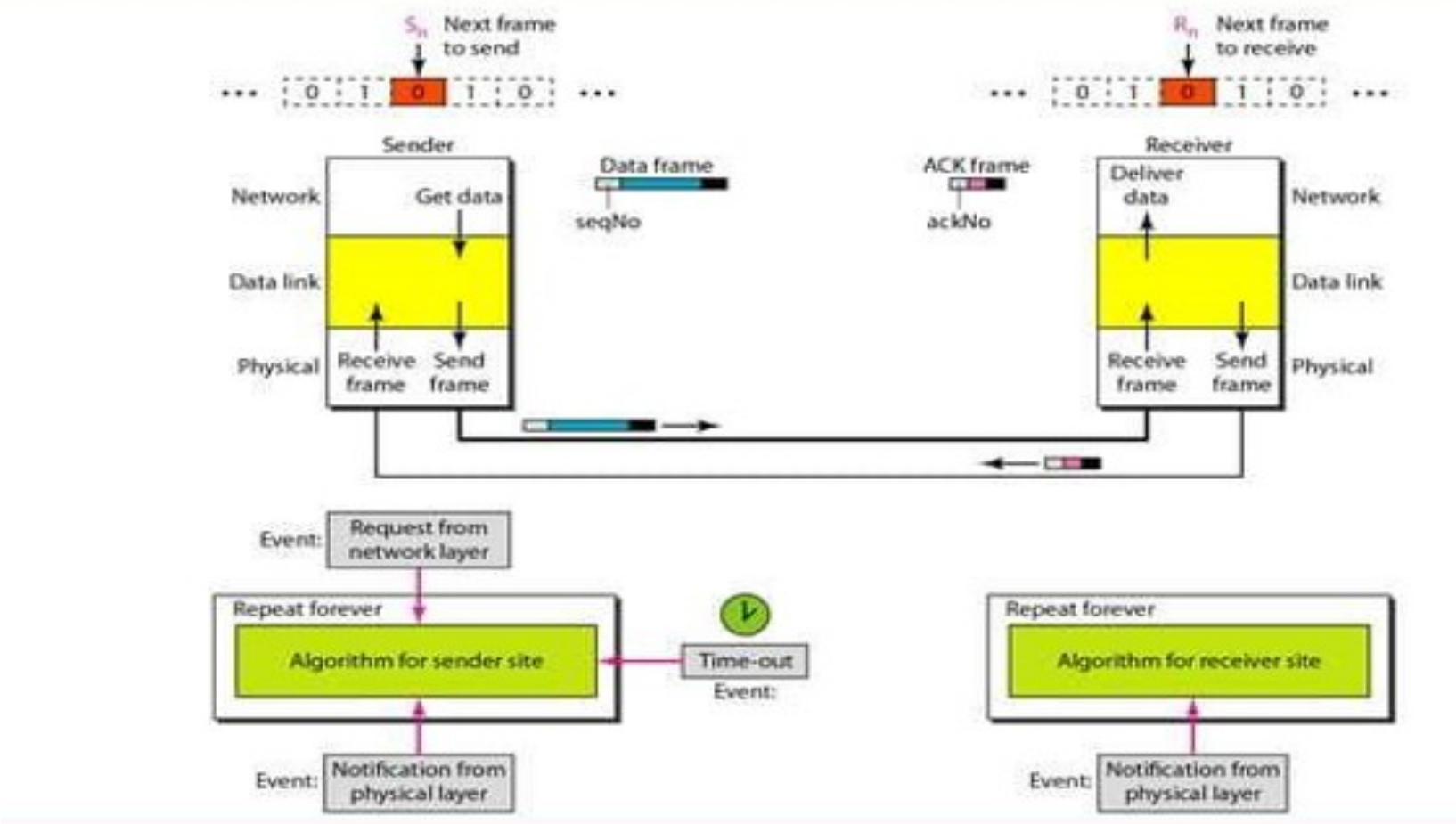
Stop and Wait with ARQ Protocol

- The completed and lost frames need to be resent in this protocol.
- If the receiver does not respond when there is an error, how can the sender know which frame to resend?
- To remedy this problem, the sender keeps a copy of the sent frame.
- At the same time, it starts a timer.
- If the timer expires and there is no ACK for the sent frame, the frame is resent, the copy is held, and the timer is restarted.

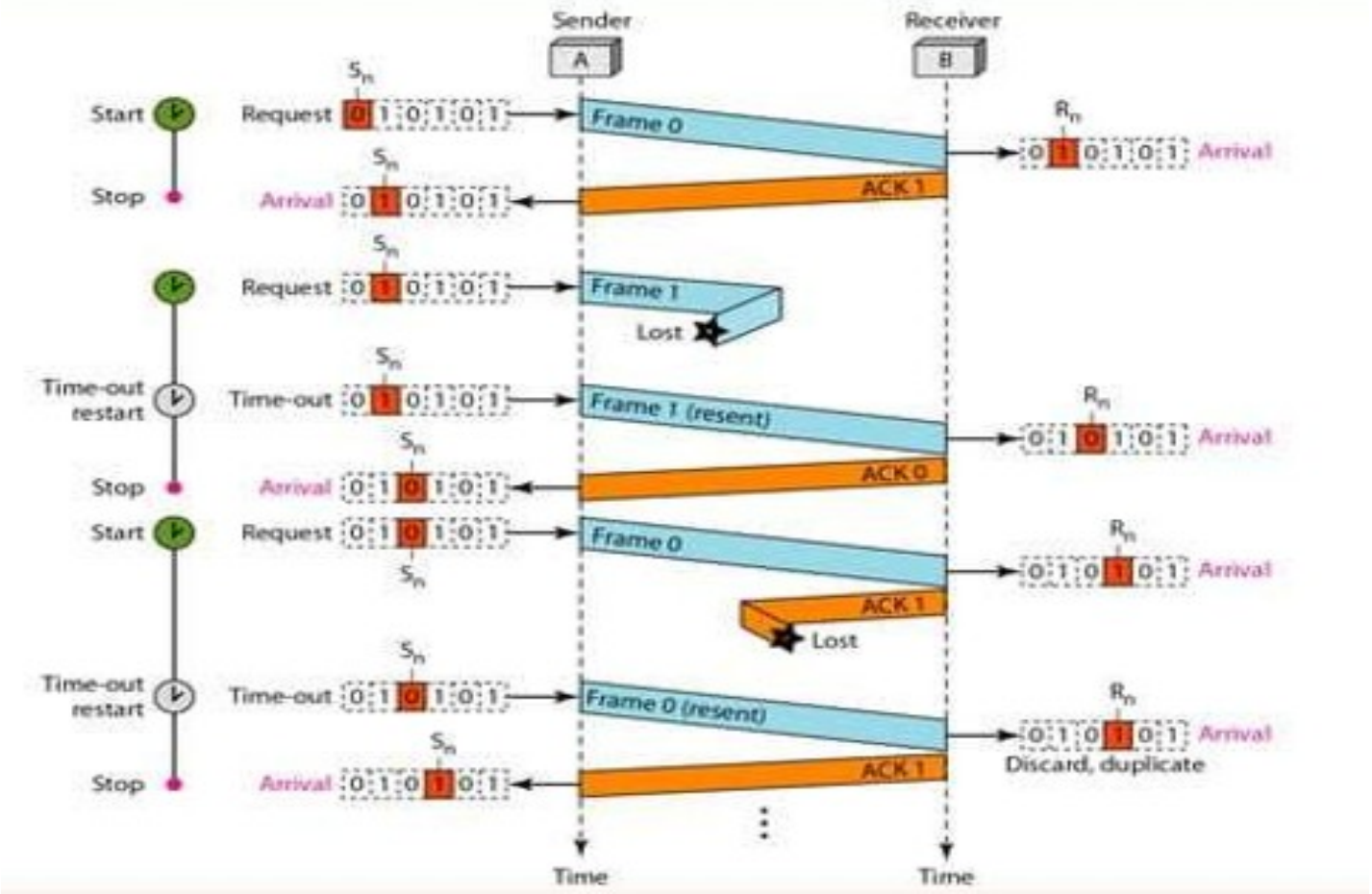
Stop and Wait with ARQ Protocol

- Since the protocol uses the stop-and-wait mechanism, there is only one specific frame that needs an ACK even though several copies of the same frame can be in the network
- Since an ACK frame can also be corrupted and lost, it too needs redundancy bits and a sequence number.
- The ACK frame for this protocol has a sequence number field.
- In this protocol, the sender simply discards a corrupted ACK frame or ignores an out-of-order one.

Design


S_n Next frame to send
0 1 0 1 0
R_n Next frame to receive
0 1 0 1 0
Sender
Network
Get data
Data link
Physical
Receive frame
Send frame
Data frame
seqNo
ACK frame
ackNo
Receiver
Deliver data
Network
Data link
Physical
Receive frame
Send frame
Event:
Request from network layer
Repeat forever
Algorithm for sender site
Time-out
Event:
Event:
Notification from physical layer
Repeat forever
Algorithm for receiver site
Event:
Notification from physical layer

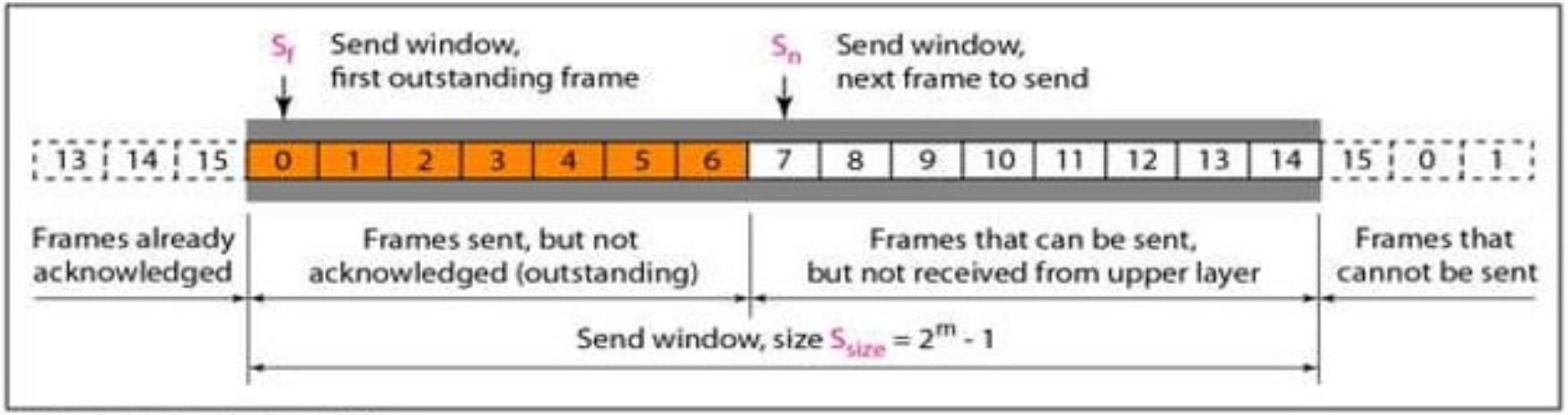
Flow Control



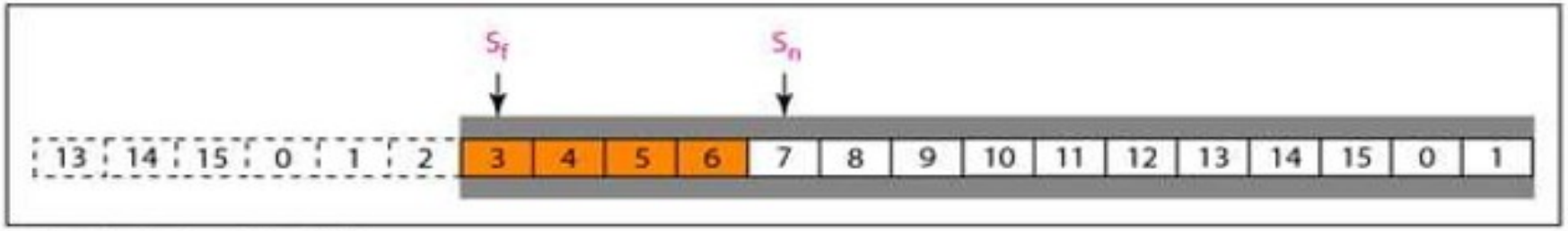
Go-Back-N With ARQ Protocol

- To improve the efficiency of transmission (filling the pipe), multiple frames must be in transition while waiting for acknowledgment.
- In other words, we need to let more than one frame be outstanding to keep the channel busy while the sender is waiting for acknowledgment.
- In this section, we discuss one protocol that can achieve this goal.
- The first is called Go-Back-N Automatic Repeat Request (the rationale for the name will become clear later).
- In this protocol we can send several frames before receiving acknowledgments; we keep a copy of these frames until the acknowledgments arrive.

Sender Window

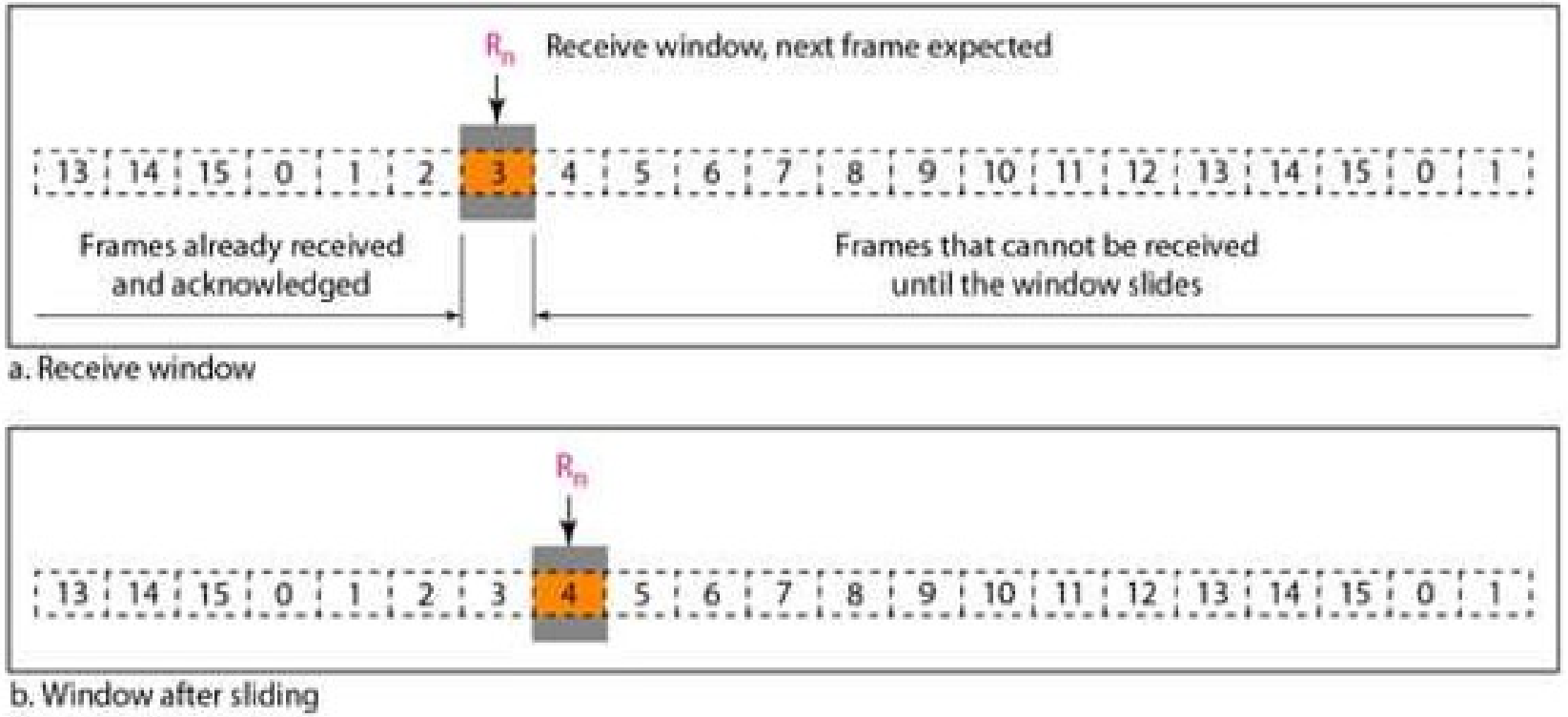


a. Send window before sliding

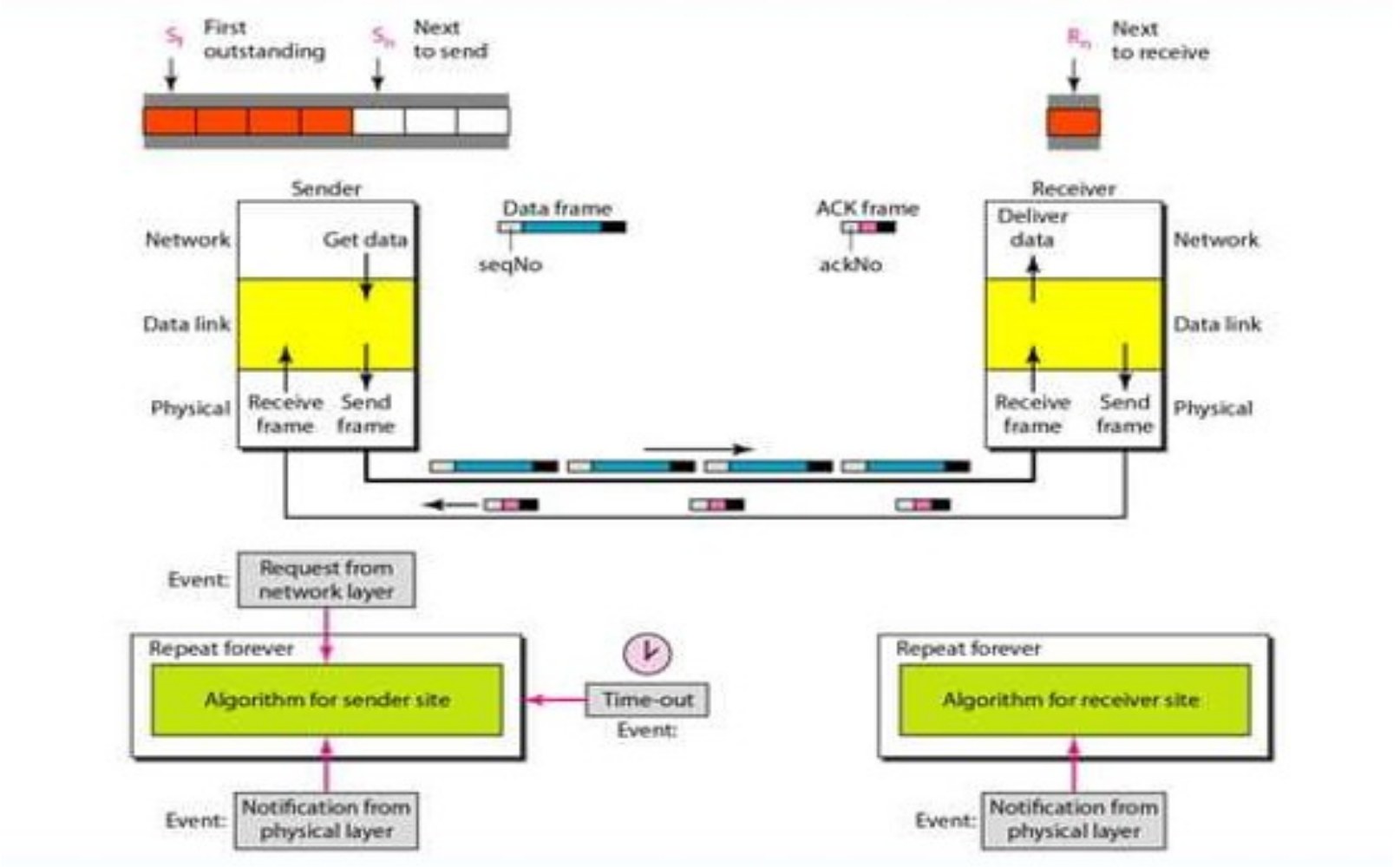


b. Send window after sliding

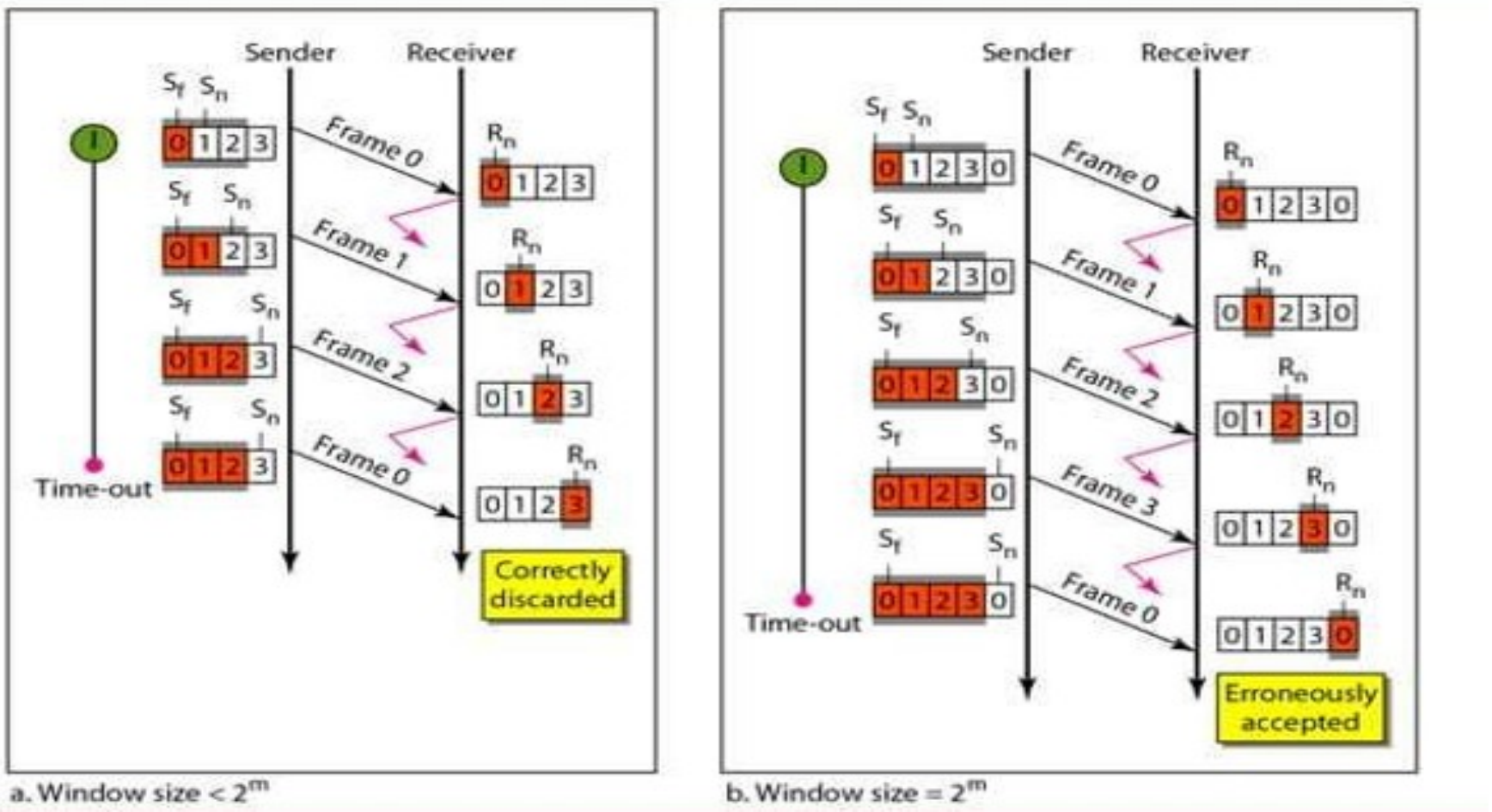
Receiver Window



Design


First outstanding
Next to send
Next to receive
Sender
Receiver
Network
Get data
Data frame
seqNo
ACK frame
ackNo
Deliver data
Network
Data link
Data link
Physical
Receive frame
Send frame
Receive frame
Send frame
Physical
Event:
Request from network layer
Repeat forever
Algorithm for sender site
Time-out
Event:
Repeat forever
Algorithm for receiver site
Event:
Notification from physical layer
Event:
Notification from physical layer

Flow


Sender
Receiver
Frame 0
Frame 1
Frame 2
Frame 0
Time-out
Correctly discarded
a. Window size < 2^m
Sender
Receiver
Frame 0
Frame 1
Frame 2
Frame 3
Frame 0
Time-out
Erroneously accepted
b. Window size = 2^m

Selective Repeat With ARQ Protocol

- Go-Back-N ARQ simplifies the process at the receiver site.
- The receiver keeps track of only one variable, and there is no need to buffer out-of-order frames; they are simply discarded.
- However, this protocol is very inefficient for a noisy link. In a noisy link a frame has a higher probability of damage, which means the resending of multiple frames.
- This resending uses up the bandwidth and slows down the transmission.
- For noisy links, there is another mechanism that does not resend N frames when just one frame is damaged; only the damaged frame is resent.

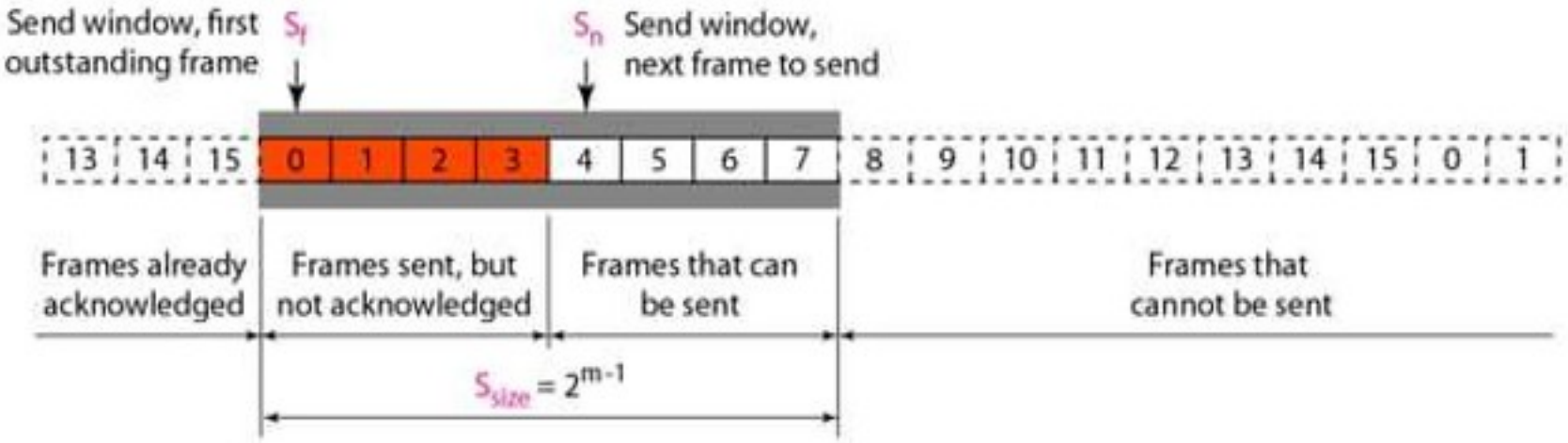
Selective Repeat With ARQ Protocol

- This mechanism is called Selective Repeat ARQ.
- It is more efficient for noisy links, but the processing at the receiver is more complex.
- The Selective Repeat Protocol also uses two windows: a send window and a receive window.
- However, there are differences between the windows in this protocol and the ones in Go-Back-N. First, the size of the send window is much smaller; it is 2m- I .
- The reason for this will be discussed later. Second, the receive window is the same size as the send window.

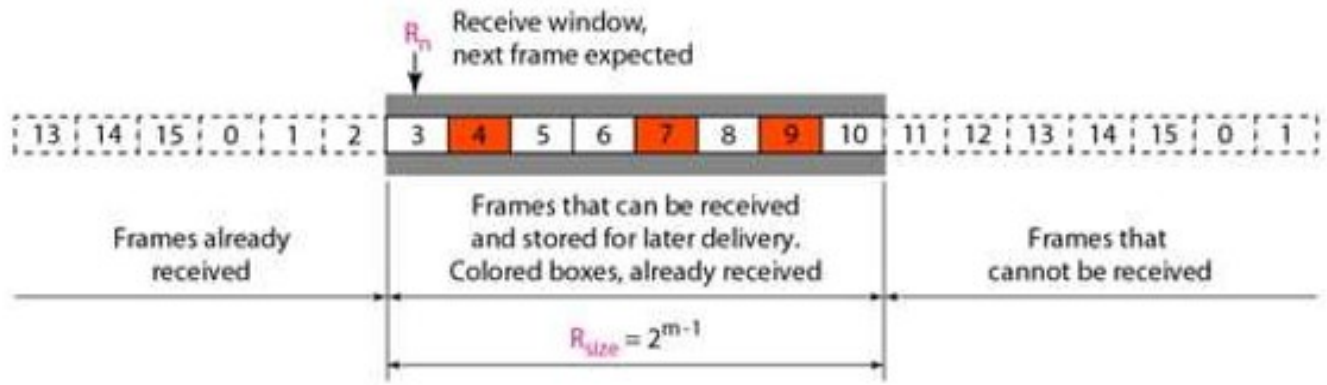
Selective Repeat With ARQ Protocol

- The send window maximum size can be 2m- I .
- For example, if m = 4, the sequence numbers go from 0 to 15, but the size of the window is just 8 (it is 15 in the Go-Back-N Protocol).
- The smaller window size means less efficiency in filling the pipe, but the fact that there are fewer duplicate frames can compensate for this.
- The protocol uses the same variables as we discussed for Go-Back-N.

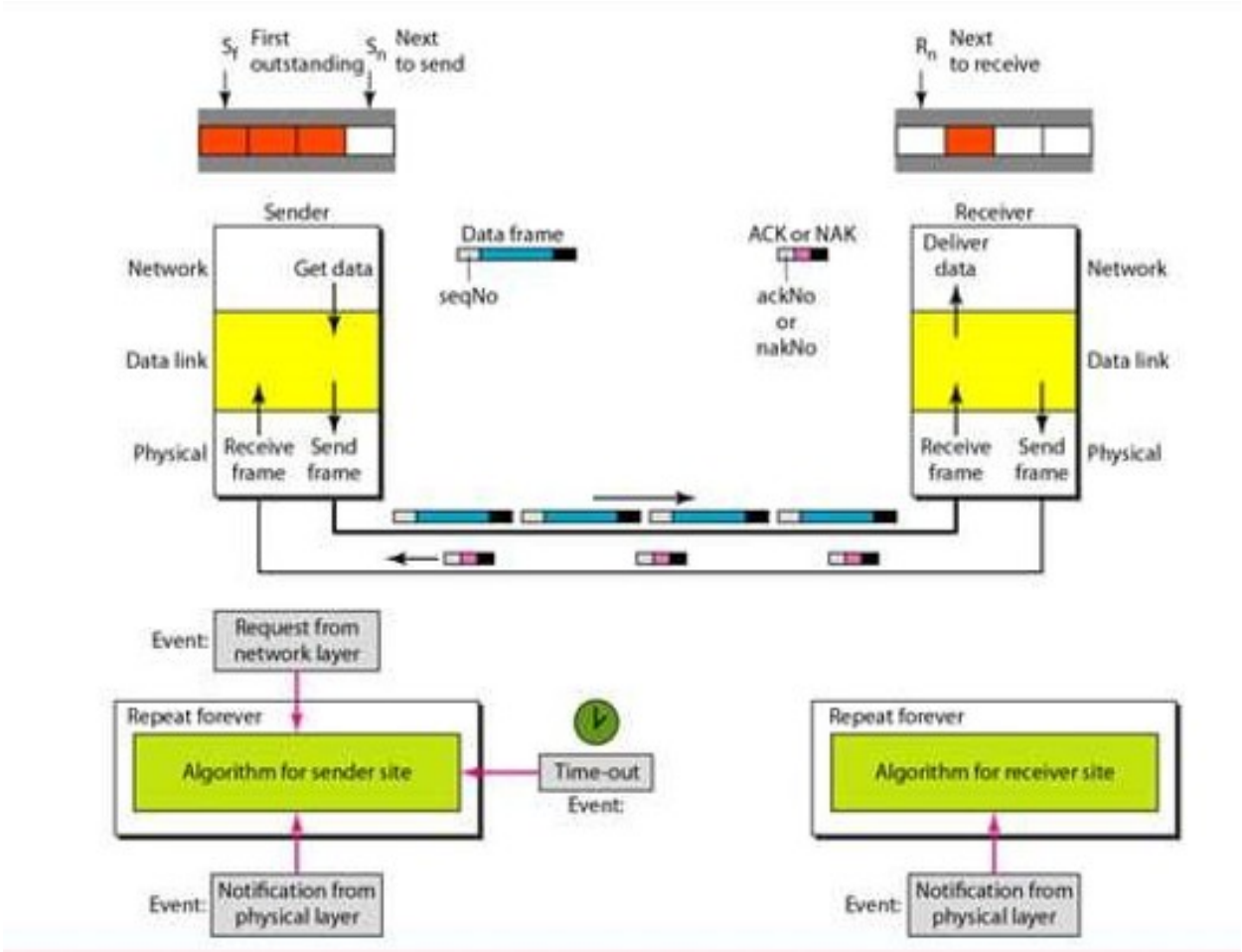
Sender Window



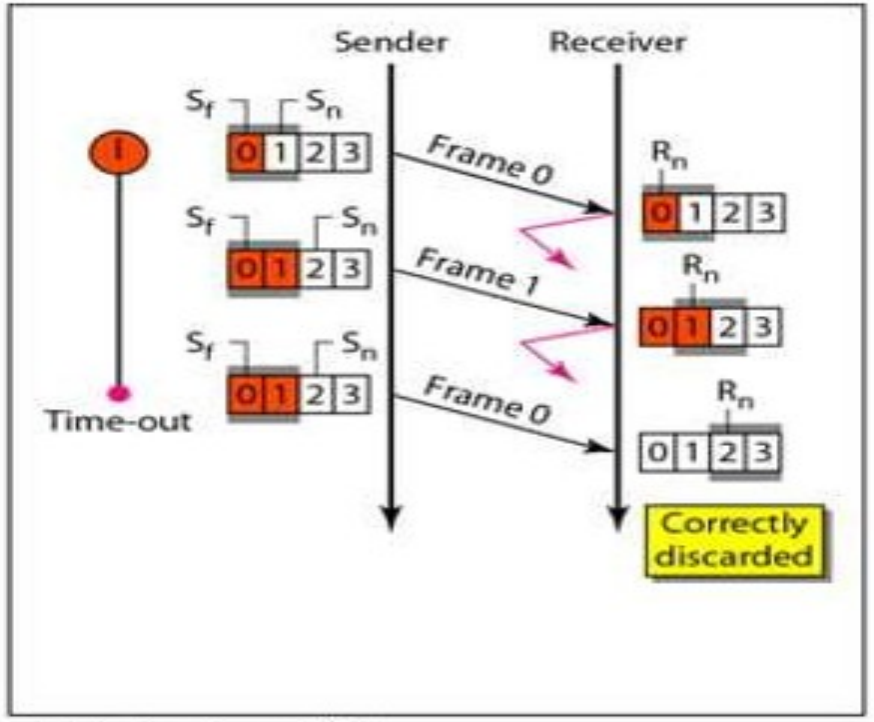
Receiver Window



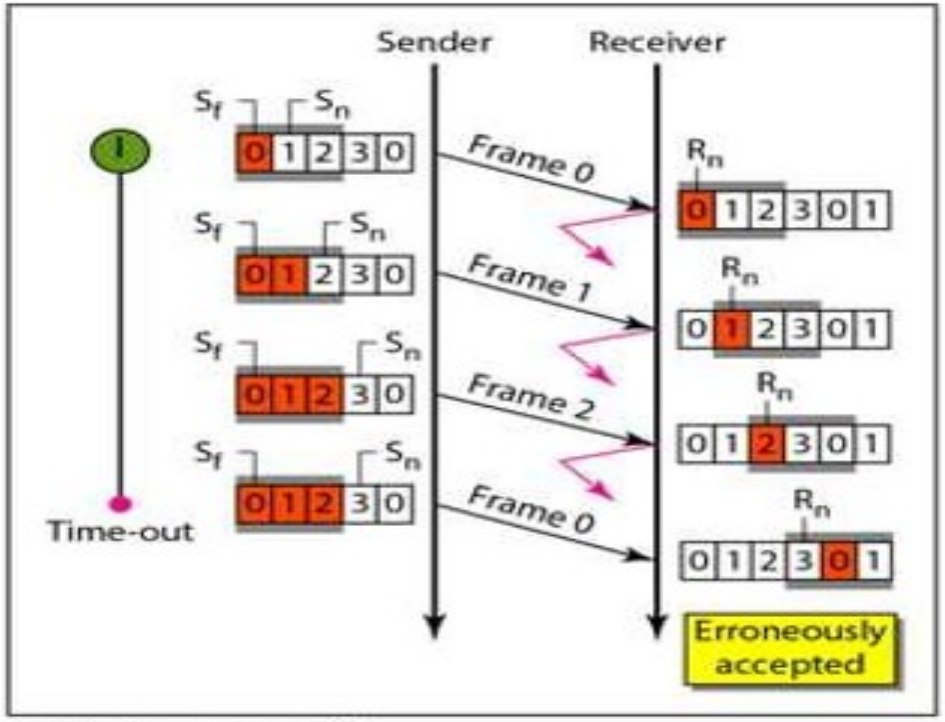
Design


First outstanding
Next to send
Next to receive
Sender
Receiver
Network
Get data
Data link
Physical
Receive frame
Send frame
Data frame
seqNo
ACK or NAK
ackNo or nakNo
Deliver data
Event:
Request from network layer
Repeat forever
Algorithm for sender site
Time-out
Event:
Notification from physical layer
Repeat forever
Algorithm for receiver site
Event:
Notification from physical layer

Flow



a. Window size = 2^{m-1}



b. Window size > 2^{m-1}